

Yatri App Roadmap Interview Guide

Offline-first Nepal travel safety companion

Scam alerts • SOS • District packs • Fair prices • AI assistant

Prepared from the project roadmap • July 6, 2026

1. One-sentence product pitch

Yatri is an offline-first Nepal travel safety companion that helps tourists avoid scams, check fair prices, save district guides, share SOS location details, and use essential cultural guidance while traveling.

2. MVP promise

The app was intentionally narrowed around one core promise:

Avoid scams and travel safely offline in Nepal.

This helped us avoid building too many broad travel features too early. The first beta focuses on five trust-building features:

- Scam alerts
- Offline district packs
- SOS and emergency contacts
- Fair price checking
- Phrases and cultural guidance

Delayed features include booking, paid local marketplace, camera recognition, carbon tracking, and professional medical advice beyond basic safety disclaimers.

3. Technology stack

Frontend:

- Expo React Native
- TypeScript
- React hooks
- React Native Web through Expo
- expo-location
- expo-sms
- expo-image-picker
- expo-network

Backend:

- Supabase Auth
- Supabase Postgres
- Supabase Row Level Security
- Supabase Storage
- Supabase Realtime
- Supabase Edge Functions

Offline/local storage:

- AsyncStorage for local preferences, saved district packs, cached location, pending reports, and trusted contacts.

Deployment:

- EAS Build config for development, preview, and production builds.
- Supabase migrations and Edge Functions stored in the repo.

4. High-level architecture

```
Expo App
|
|-- Auth/session -> Supabase Auth
|-- Reports/votes/profiles -> Supabase Postgres with RLS
|-- Photos -> Supabase Storage bucket: report-photos
|-- Live report updates -> Supabase Realtime
|-- Account deletion -> Supabase Edge Function: delete-account
|-- AI assistant -> Supabase Edge Function: travel-assistant
|-- Offline queue/cache -> AsyncStorage
|-- GPS/SOS -> expo-location + expo-sms
```

The app is designed so public client keys can be shipped safely, while privileged keys stay in Supabase Edge Function secrets.

5. Build roadmap: how we built the app

Phase 1: Product narrowing

We reduced the product scope. Instead of trying to be a complete travel marketplace, Yatri became a reliable MVP around safety and offline trust.

Key decision:

- Focus on scam alerts, offline packs, SOS, fair prices, and phrases.
- Delay booking, payments, paid locals, camera recognition, and professional medical advice.

Interview talking point:

I started by narrowing the MVP around trust. For a travel safety product, reliability, privacy, and offline usefulness matter more than having a huge feature list.

Phase 2: Real authentication

We replaced demo/local login with Supabase authentication.

Implemented:

- Email/password sign up
- Email/password sign in
- Password reset
- Session restore
- Sign out
- Guest continue flow
- Supabase configuration guard if env keys are missing

Important files:

- App.tsx
- src/auth/supabase.ts
- src/screens/YatriLoginScreen.tsx

Interview talking point:

I used Supabase Auth because it gives secure password handling, session refresh, and email auth without building custom auth infrastructure. The app only stores a public publishable/anon key, never service-role secrets.

Phase 3: Backend schema and security

We added a Supabase migration for the safety MVP.

Tables added:

- profiles
- scam_reports
- report_votes
- saved_districts
- trusted_contacts
- content_items

Storage added:

- report-photos bucket

Security added:

- Row Level Security enabled
- Users can manage their own profile, saved districts, contacts, and votes
- Authenticated users can insert reports
- Public users can read non-rejected reports
- Moderator/admin role can verify or reject community reports

Important file:

- supabase/migrations/202607050001_safety_mvp.sql

Interview talking point:

I separated community reports from verified alerts. A report starts as community content and only becomes a verified alert after moderation. That reduces misinformation risk.

Phase 4: Real location and SOS

We added foreground-only location using expo-location.

Implemented:

- Foreground location request only
- Last-known GPS fallback
- Cached last GPS fix in AsyncStorage
- Accuracy and age display
- SOS SMS composer using expo-sms
- Trusted and embassy contact storage
- Nepal Tourist Police 1144 call action

Important files:

- src/services/location.ts
- src/screens/YatriDashboardScreen.tsx

Privacy decision:

- No background location.
- Location is used only while the app is open.

Interview talking point:

I intentionally avoided background location because the MVP only needs foreground GPS for nearby alerts and SOS. That keeps privacy risk and app store review risk lower.

Phase 5: Production-style scam reporting

We upgraded scam reporting from static demo pins to live report handling.

Implemented:

- Real GPS coordinates
- User descriptions
- Optional report photos
- Offline report queue
- Sync later when online
- Duplicate detection window
- Voting / "I saw this too"
- Realtime report subscription
- Community vs verified status
- Moderator review panel

Important files:

- src/services/mvpRepository.ts
- src/screens/YatriDashboardScreen.tsx
- Supabase migration

Interview talking point:

I designed the report flow to work offline first. If the user has no network, reports are queued locally and synced later. For a travel safety app, unreliable connectivity is a core use case.

Phase 6: Offline district packs

We made district selection and offline saving more useful.

Implemented:

- Kathmandu selected by default
- Full searchable list of all 77 districts of Nepal
- Search by district or province
- Rich guide cards for districts with detailed content
- Starter guide cards for districts not fully expanded yet
- Offline save/download state
- District-specific lodging search cards

Important files:

- src/data/yatriData.ts
- src/screens/YatriDashboardScreen.tsx

Interview talking point:

I separated the complete district directory from rich guide content. That lets the UI support all 77 districts now, while content depth can grow over time through a CMS/admin workflow.

Phase 7: Fair price checker

We improved the price feature from static lists into a practical fair-price tool.

Implemented:

- Searchable price catalog
- Category filters for Food, Transport, Shopping, Permits, and Connectivity
- Quoted price input
- Verdicts: within range, slightly high, likely overcharge, or official-check-needed
- Risk labels
- Nepali bargaining/help phrases
- Source/freshness note

Important files:

- src/data/yatriData.ts
- src/screens/YatriDashboardScreen.tsx

Interview talking point:

The price checker is not presented as an official tariff. It is clearly labeled as a community reference and encourages users to verify locally. That keeps the UX helpful without overclaiming accuracy.

Phase 8: Festivals and local culture

We improved festival content so the app does not skip important festivals before Dashain.

Added 2026 pre-Dashain festival timeline:

- Janai Purnima / Raksha Bandhan
- Gai Jatra
- Gaura Parba
- Krishna Janmashtami
- Haritalika Teej
- Rishi Panchami
- Indra Jatra
- Ghatasthapana
- Vijaya Dashami
- Tihar

Important file:

- src/data/yatriData.ts

Interview talking point:

Festival dates are temporal content, so the app shows a review date and should eventually be managed through CMS content rather than hardcoded app releases.

Phase 9: AI assistant

We added a floating Yatri AI chat assistant after login/dashboard.

Implemented:

- Floating chat UI
- Available on main app pages, not login
- Local fallback answers for safety, scams, SOS, prices, offline prep, and etiquette
- Supabase Edge Function proxy: travel-assistant
- Secret-based AI endpoint configuration

Important files:

- src/components/YatriAiChat.tsx
- src/services/aiAssistant.ts
- supabase/functions/travel-assistant/index.ts

Security decision:

- Private AI model keys are not placed in the Expo app.
- The app calls a Supabase Edge Function.
- The Edge Function calls the trained model endpoint using Supabase secrets.

Interview talking point:

I avoided putting the AI key in the mobile app because anything bundled into a client can be extracted. The app uses an Edge Function proxy so model credentials stay server-side.

Phase 10: Web-friendly layout

We adjusted the Expo web build because mobile typography felt small on desktop.

Implemented:

- Larger desktop hero text
- Larger page and section headings
- Wider content container
- Desktop-friendly login layout
- Larger AI chat panel on web
- Larger district and price text

Important files:

- src/screens/YatriLoginScreen.tsx
- src/screens/YatriDashboardScreen.tsx
- src/components/YatriAiChat.tsx

Interview talking point:

Since Expo targets both mobile and web, I used responsive width checks to keep compact mobile layout while giving desktop users larger typography and more spacious layouts.

6. Core files and what they do

App.tsx

Controls top-level app state:

- loading
- login
- preferences
- location
- dashboard

Also restores Supabase sessions and renders the AI chat only on dashboard.

src/auth/supabase.ts

Creates the Supabase client and validates environment config.

Uses:

- EXPO_PUBLIC_SUPABASE_URL
- EXPO_PUBLIC_SUPABASE_ANON_KEY

src/auth/localSession.ts

Stores local onboarding and preferences.

src/services/location.ts

Handles foreground GPS and cached last location.

src/services/mvpRepository.ts

Repository/service layer for reports, votes, saved districts, trusted contacts, moderation, account deletion, and offline sync.

src/services/aiAssistant.ts

Client-side AI helper. Calls Supabase Edge Function when available; otherwise returns safe fallback answers.

src/screens/YatriLoginScreen.tsx

Auth UI for sign in, sign up, reset password, and guest mode.

src/screens/YatriDashboardScreen.tsx

Main app UI: Home, Explore, Safety, Local, Prices, and Moderation.

src/data/yatriData.ts

Seed/local MVP content: district directory, district guide content, prices, festivals, phrases, etiquette, hotels, and discovery content.

supabase/migrations/202607050001_safety_mvp.sql

Database schema, RLS, storage, and realtime setup.

supabase/functions/delete-account/index.ts

Deletes the authenticated user account server-side.

supabase/functions/travel-assistant/index.ts

Proxies AI requests to the trained model endpoint.

7. Environment variables

Expo app, stored in .env:

```
EXPO_PUBLIC_SUPABASE_URL=https://xugaakqczxhrjzbnulut.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your_publishable_or_anon_key
EXPO_PUBLIC_PRIVACY_URL=https://example.com/privacy
EXPO_PUBLIC_TERMS_URL=https://example.com/terms
EXPO_PUBLIC_SUPPORT_URL=https://example.com/contact
```

Only public Expo variables should go here.

Supabase Edge Function secrets:

```
npx supabase secrets set YATRI_AI_ENDPOINT=https://your-trained-model-endpoint
npx supabase secrets set YATRI_AI_API_KEY=your_private_model_key
npx supabase secrets set YATRI_AI_MODEL=your_model_name
```

Do not put service-role keys or private AI keys in the mobile app.

8. Supabase commands used

```
npx supabase login
npx supabase link --project-ref xugaakqczxhbjbnulut
npx supabase db push --include-seed
npx supabase functions deploy delete-account --project-ref xugaakqczxhbjbnulut
npx supabase functions deploy travel-assistant --project-ref xugaakqczxhbjbnulut --no-verify-jwt
```

9. Testing and verification

Current checks:

- TypeScript check passes
- MVP test file exists under tests/mvp.test.mjs
- App config uses foreground-only location
- Migration includes RLS and moderation paths
- Package config includes required Expo/Supabase dependencies

Recommended next tests:

- Test signup/signin on web and phone
- Test password reset email
- Test foreground location on physical Android/iOS devices
- Test SMS composer on physical devices
- Test offline report queue by disabling network
- Test report sync after network returns
- Test Supabase RLS manually with anon/authenticated users
- Test moderator role flow
- Test account deletion Edge Function
- Test AI assistant with real trained model endpoint

10. Privacy and safety decisions

Yatri handles sensitive contexts, so the MVP includes privacy-aware choices:

- No background location
- No automatic SOS sending
- SOS message is prepared by the user
- Contacts are user-controlled
- Reports are marked community until verified
- AI chat warns against entering sensitive personal info
- Medical/altitude feature is informational only, not a diagnosis
- Service-role and AI secrets stay server-side
- Privacy, terms, and beta checklist docs exist

11. App store readiness checklist

Before public store launch:

- Publish real privacy policy URL
- Publish real terms URL
- Add support/contact page
- Configure account deletion fully
- Fill Apple privacy nutrition labels
- Fill Google Data Safety form
- Verify location permission copy
- Verify no background location permission is requested
- Add crash/error monitoring
- Add analytics with privacy controls
- Add abuse controls for scam reports
- Run TestFlight and Google Play internal beta
- Test with real tourists and Nepali guides
- Build using EAS

12. Likely interview questions and strong answers

Q1. What problem does Yatri solve?

Yatri helps tourists in Nepal avoid scams and travel safely when internet access is unreliable. It combines scam reports, fair price checks, offline district guides, SOS location sharing, phrases, and cultural guidance.

Q2. Why did you narrow the scope?

Travel apps can become too broad. We focused on the highest-trust, highest-need use case: safety offline. Features like booking, paid locals, and camera recognition were delayed because they require partnerships, payments, moderation, or professional review.

Q3. Why Supabase?

Supabase provides authentication, Postgres, RLS, storage, realtime, and Edge Functions in one stack. That made it fast to move from demo/local data to a real backend while still keeping security controls like RLS.

Q4. How did you secure user data?

We used Supabase Auth and Row Level Security. Users can only manage their own profiles, contacts, saved districts, and votes. Report moderation is limited to moderator/admin roles. Private keys stay in Edge Function secrets, not the app.

Q5. How does offline support work?

The app stores important data locally with AsyncStorage. District packs, preferences, trusted contacts, and last GPS fix are available offline. Scam reports can be queued locally and synced later when the network returns.

Q6. How does scam reporting avoid misinformation?

Reports start as community reports, not verified alerts. Other users can vote/confirm, but moderators decide whether a report becomes verified or rejected. This separates raw community signals from trusted alerts.

Q7. How does location work?

The app requests foreground location only. It uses GPS while the app is open for nearby reports and SOS message preparation. It does not request background location.

Q8. Why not automatically send SOS messages?

Automatic SOS can create false alarms and privacy risks. Yatri prepares the SMS with GPS details, but the user confirms sending through their phone's native SMS composer.

Q9. How did you integrate AI safely?

The app has a floating AI assistant, but it does not store private model keys. It calls a Supabase Edge Function, and that function calls the trained model endpoint using Supabase secrets. There is also a local fallback for basic travel safety answers.

Q10. How is the price checker designed?

It uses a catalog of common tourist prices, categories, risk labels, and phrase guidance. Users can enter a quoted price, and Yatri classifies it as within range, slightly high, likely overcharge, or official-check-needed. It is labeled as community reference, not an official tariff.

Q11. How does the district feature scale to all Nepal?

The app has a complete 77-district directory. Some districts have rich guide content now, and others use starter cards. This lets the UI support all districts immediately while richer content can be added over time.

Q12. What are the biggest technical challenges?

- Offline-first syncing
- Location permissions and privacy
- Moderating user-generated safety reports
- Keeping price/festival content accurate
- Avoiding client-side exposure of private AI keys
- Making a mobile-first Expo app also feel good on web

Q13. What would you improve next?

- Add CMS/admin dashboard for content updates
- Add real map tiles or MapView implementation
- Add stronger duplicate report detection
- Add push notifications for verified alerts
- Add report abuse controls
- Add richer district content for all 77 districts
- Add real source-backed official price/permit feeds where available
- Add analytics and crash monitoring
- Add automated integration tests

Q14. What tradeoffs did you make?

We chose MVP reliability over feature breadth. Some content is currently local/static, but the schema already supports sourced content and moderation. We also use starter district cards for full coverage while detailed local content grows over time.

Q15. How would you explain the architecture in 30 seconds?

Yatri is an Expo React Native app backed by Supabase. Supabase handles auth, Postgres, RLS, storage, realtime reports, and Edge Functions. The app caches key data locally for offline use, queues reports when offline, uses foreground GPS only, and keeps privileged AI/service keys server-side.

13. Demo script for an interview

- 1. Start at login and explain Supabase Auth.
- 2. Sign in or continue as guest.
- 3. Show Home page and offline/online toggle.
- 4. Search a district, for example Achham.
- 5. Show district guide and district-specific lodging searches.
- 6. Go to Prices and search taxi or SIM.
- 7. Enter an inflated quote and show overcharge warning.
- 8. Go to Safety and show scam alert map/report flow.
- 9. Show SOS with GPS refresh and SMS preparation.
- 10. Open Yatri AI and ask: Is Rs. 1500 too high from airport to Thamel?
- 11. Explain that the AI key is protected behind Supabase Edge Functions.
- 12. Mention moderation and community vs verified alerts.

14. Short resume bullet version

- Built an offline-first Nepal travel safety app using Expo React Native, TypeScript, and Supabase.
- Implemented Supabase Auth, Row Level Security, realtime scam reports, report voting, moderation, photo storage, and account deletion.
- Added foreground-only GPS, SOS SMS preparation, trusted contacts, offline report queueing, and district guide downloads.
- Built a searchable 77-district Nepal directory with district-specific lodging guidance and offline save support.
- Designed a fair-price checker with category filters, quote comparison, overcharge warnings, and Nepali negotiation phrases.
- Integrated a floating AI assistant through a Supabase Edge Function proxy to protect private model credentials.
- Improved Expo web responsiveness with desktop-friendly typography, layout, and chat UI.

15. Current limitations to be honest about

- Some price references are community estimates, not official tariffs.
- Not every district has rich local guide content yet.
- Real map tiles/native map rendering can still be improved.
- AI model endpoint must be configured through Supabase secrets.
- Physical-device testing is still required for SMS, phone calls, and location accuracy.
- A production CMS/admin dashboard should be added before large-scale launch.

16. Best closing statement

Yatri is built as a focused MVP, not a feature dump. The app prioritizes user trust: secure accounts, foreground-only location, offline safety tools, moderated scam reports, transparent price references, and server-side protection for AI credentials. The architecture is ready to grow, but the first beta stays centered on helping travelers avoid scams and stay safe offline in Nepal.